Universidad
Complutense & Politécnica
de Madrid

Master's in Formal Methods in Computer Science

Master's Thesis

Type Theory and Categorical Models
A Unified Approach

Miguel Laredo Barbadillo
Raúl Lara Cabrera
Guillermo Román Díez

Madrid, August 24, 2025

This Final Degree Project has been submitted to the ETSI Informáticos of the Universidad Politécnica de Madrid for its defense.

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Master's in	Formal Methods in Computer Science
Title	Type Theory and Categorical Models: A Unified Approach

Author	Miguel Laredo Barbadillo
Tutors	Raúl Lara Cabrera
	Dpto. Sistemas Informáticos
	Guillermo Román Díez
	Dpto. Lenguajes y Sistemas Informáticos e Ingeniería de Softwaree

Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid

Gracias.

Contents

1 Untyped λ-Calculus **11**
1.1 The Language of λ-Calculus 12
1.2 Reduction of terms 13
1.3 Normalization & The Church-Rosser Theorem 21
1.4 Relevant Constructions 24
1.5 Fixed Point Combinators and Recursion 28

2 Typing **33**
2.1 Minimal Typing 33
2.2 Extended Typing 36
2.3 Normalization in Minimal and Extended Typing 37

3 Intuitionistic Logic & Curry–Howard **39**
3.1 Intuitionistic Logic 39
3.2 The Curry-Howard Isomorphism 41

4 Introduction to Category Theory **43**
4.1 Fundamentals of Category Theory 43

5 Curry–Howard–Lambek & Categorical Semantics **53**

Abstract

This work investigates the interplay of lambda calculus, type theory, category theory, through the Curry-Howard-Lambek correspondence. It explores lambda calculus as a computational foundation, its type-safe expression, and its categorical semantics.

Esta trabajo examina la interacción entre el cálculo lambda, la teoría de tipos y la teoría de categorías, a través de la correspondencia Curry-Howard-Lambek. Explora el cálculo lambda como base computacional, su expresión tipada y su semántica categórica.

Keywords: Lambda Calculus, Type Theory, Category Theory, Curry-Howard-Lambek Correspondence.

Introduction

This work explores the untyped and typed lambda calculus, intuitionistic logic, and the Curry–Howard Isomorphism. It then expands the correspondence adding categorical semantics into what is referred to as the Curry–Howard–Lambek Correspondence.

External Resources

To clone the code used to implement the λ-calculus:

```
$ git clone https://github.com/laredo02/lambda-interpreter.git
```

To clone the LaTeX source used to generate this document:

```
$ git clone https://github.com/laredo02/ttcmua.git
```

Chapter 1

Untyped λ-Calculus

In the early 20th century, mathematicians such as Bertrand Russell, David Hilbert, and Kurt Gödel were trying to set complete, consistent, and decidable foundations for mathematics. Within this context, Alonzo Church, in his effort to formalize the notion of computability, introduced a minimal symbolic language based upon function abstraction and function application, this system is now called λ-calculus.

In 1936, Church used lambda calculus to address Hilberts Entscheidungsproblem –whether a mechanical method could determine the truth of any first-order logic statement. He proved no such algorithm exists, establishing the undecidability of first-order logic.

One of the nuances formalized by the λ-calculus is the distinction between extensional and intensional equality. The extensional approach to equivalence states that two functions are equivalent if they share input output pairs for every possible input. Its intensional counterpart extends this notion of equality by requiring that the procedures that compute these pairs share complexity i.e. they take the same steps toward yielding a result.[1]

The Lambda Calculus together with Type Theory lay the theoretical groundwork that shaped the design of languages like LISP, Haskell and Erlang to name a few. More advanced type systems than those present in general purpose programming languages enable computers to verify mathematical statements and produce rigorous proofs.

The Intuitive Notion

In order to get acquainted with the λ-calculus, let us develop a simple example to familiarize ourselves before we begin with a more formal approach to this discipline. Consider the function $f(x) = x + 1$, the most straightforward way to

[1] Let p be a sufficiently large prime, and let $f, g : \mathbb{Z}_p \to \mathbb{Z}_p$ be defined by $f(x) = x^2$ and $g(x) = \log_a(a^{x+2})$, where $a \in \mathbb{Z}_p^\times$ is a fixed primitive root. Although f and g are extensionally equal, i.e., they yield the same output for all inputs in $\mathbb{Z}_p$, they are intensionally distinct. The function f performs a simple squaring operation, while g requires evaluating a discrete logarithm, intractable in general. (Nadal 2021)

express this using λ-calculus would be $(\lambda x.x+1)$, where the lambda denotes that x is being captured and used as a parameter to perform some computation. Evaluating $f(2)$ using this newly created lambda term would look like this: $(\lambda x.x+1)(2) \rightsquigarrow (2+1) \rightsquigarrow 3$. This process of reducing a λ-expression is referred to as β-reduction, it will be covered formally later in the chapter.

1.1 The Language of λ-Calculus

Definition 1.1.1. The set of all lambda terms Λ is defined inductively as follows:

- (Variable) If $x \in \mathcal{V}$, then $x \in \Lambda$.
- (Abstraction) If $x \in \mathcal{V}$ and $M \in \Lambda$, then $(\lambda x.M) \in \Lambda$.
- (Application) If $M, N \in \Lambda$, then $(MN) \in \Lambda$.

Where $\mathcal{V} = \{x, y, z, ...\}$ represents a countably infinite set of variable names. These notions are conveniently synthesized into the following abstract syntax:

$$\Lambda := \mathcal{V} \mid \lambda x.\Lambda \mid \Lambda\,\Lambda$$

that directly translates into the following Haskell snippet:

```
data Term = Var String
          | Abs String Term
          | App Term Term
```

The key takeaway of this definition is that abstraction and application when paired with a reduction procedure, capture the meaning of function in a way that is abstract yet useful, and enables computation.

Notation 1. We use uppercase letters, e.g. M, N, L, P, Q to denote elements of Λ and lowercase e.g. x, y, z to denote elements of $\mathcal{V}$.

Remark 1. From the definition above we extract that $\lambda x.x+1 \notin \Lambda$, given this circumstance, the need to find an accurate representation for numbers within this definition arises. See Definition 1.4.3

Example 1.1.1. Some examples of valid λ terms generated using the syntax from 1.1.1: y, $(\lambda x.(xx))$, $(\lambda x.(\lambda y.x))$, $(((\lambda x.(xy))(\lambda y.y))z)$. The Haskell counterpart for the second term is:

```
Abs "x" (Abs "y" (Var "x"))
```

As a convention to avoid notational cluttering, the outermost parenthesis can be omitted e.g. $\lambda x.x$ is read as $(\lambda x.x)$. Also, application is left-associative and binds tighter than abstraction, so MNP is parsed as $(MN)P$ and $\lambda x.MN$ means $\lambda x.(MN)$, not $(\lambda x.M)N$. Likewise, abstraction is right-associative e.g. $\lambda x.\lambda y.M$ is $\lambda x.(\lambda y.M)$ and binds more weakly than application. Haskell Curry introduced what we now refer to as currying, its main purpose is to avoid unnecessary lambdas. Below is a summary of common notational conventions used throughout the text:

- $\lambda x.x \iff (\lambda x.x)$
- $MNP \iff (MN)P$
- $\lambda x.MN \iff \lambda x.(MN)$
- $\lambda x.\lambda y.f(x,y) \iff \lambda xy.f(x,y)$

All these are just to keep notation straight-forward, since the formal definition does not leave precedence under-specified thanks to parenthesis. Those interested in the implementation need not worry, as Haskell enforces an explicit order on constructors.

1.2 Reduction of terms

Now that the λ-calculus language has been introduced, it is time we move on to the mechanics of computation, manifested through β-equivalence. α and η equivalence will also be introduced. These two are more difficult to grasp. Informally, it could be said that α-equivalence is equivalence programs up to renaming of local variables. In this same scenario, η-equivalence would singnify equivalence of programs up to wrapping or unwrapping them with an identity function.

Subterms

From the ground up, we define what a subterm is, since the definition of the language is self-referential and terms might contain other terms.

Definition 1.2.1. Given $T \in \Lambda$, $\mathbf{Sub} : \Lambda \to \mathscr{P}(\mathscr{V})$ maps a term to the set of it's subterms.

$$\begin{aligned}
&\mathbf{Sub}(x) = \{x\} \\
&\mathbf{Sub}(MN) = \mathbf{Sub}(M) \cup \mathbf{Sub}(N) \cup \{MN\} \\
&\mathbf{Sub}(\lambda x.M) = \mathbf{Sub}(M) \cup \{\lambda x.M\}
\end{aligned}$$

Where $x \in \mathscr{V}$ and $M, N \in \Lambda$. In Haskell:

```
subTerms :: Term -> Set.Set Term
subTerms (Var x) = Set.singleton (Var x)
subTerms (App m n) =
    Set.insert (App m n) (Set.union (subTerms m) (subTerms n))
subTerms (Abs x m) =
    Set.insert (Abs x m) (subTerms m)
```

Remark 2. Let $\preceq$ be the subterm relation on λ-terms, defined by

$$M \preceq N \iff M \in \mathbf{Sub}(N)$$

$\preceq$ is a partial order relation. This statement is follows intuitively from the definition, it can be proven using induction on the derivation tree.

Definition 1.2.2. A term $S \in \mathbf{Sub}(M)$, $M \in \Lambda$ is said to be proper if $S \neq M$. $\mathbf{Sub}(M) \setminus \{M\}$ would be the set of proper subterms.

Definition 1.2.3. Let $M = \lambda x.N$ be a λ-term:

- The variable x is the binding variable of the abstraction $\lambda x.N$.
- An occurrence of a variable x in M is bound if $x \in \mathbf{Sub}(\lambda x.N)$.
- Free variables are those that are not bound.

Example 1.2.1. Take $M \equiv \lambda x.\, x\, y$, in this example, the x in $\lambda x.$ is binding, y is free, and x is bound. This means that y is used literally, and that x, as denoted by $\lambda x.$ is abstracted and thus subject to being replaced within the context of said abstraction. A subexample might make this explicit: $(\lambda x.\, x\, y)\, M \rightsquigarrow_\beta M\, y$. The y remained while the x was used as a placeholder for M.

Note 1. This taxonomy is key to understanding concepts such as α-equivalence intuitively.

Free Variables

The set of Free Variables **FV**, those that will not get replaced during the process of reduction, is computed using:

Definition 1.2.4. Given $T \in \Lambda$, $\mathbf{FV} : \Lambda \to \mathscr{P}(\mathscr{V})$ outputs the set of free variables for T:

$$\begin{aligned}
\mathbf{FV}(x) &= \{x\} \\
\mathbf{FV}(MN) &= \mathbf{FV}(M) \cup \mathbf{FV}(N) \\
\mathbf{FV}(\lambda x.M) &= \mathbf{FV}(M) \setminus \{x\}
\end{aligned}$$

Where $x \in \mathscr{V}$ and $M, N \in \Lambda$, and $\mathscr{P}$ denotes the power set. In Haskell:

```
freeVars :: Term -> Set.Set String
freeVars (Var x) = Set.singleton x
freeVars (App m n) = Set.union (freeVars m) (freeVars m)
freeVars (Abs x m) = Set.delete x (freeVars m)
```

Example 1.2.2. Computing the free variables of $\lambda x.\lambda y.yxz$. The term well-formed, so we proceed:

$$\begin{aligned}
\mathbf{FV}(\lambda x.\lambda y.yxz) &= \mathbf{FV}(\lambda y.yxz) \setminus \{x\} \\
&= \mathbf{FV}(yxz) \setminus \{x, y\} \\
&= \{x, y, z\} \setminus \{x, y\} \\
&= \{z\}
\end{aligned}$$

Of course, z is the only free variable in the expression as it is the only variable that is not captured by some λ-abstraction.

Definition 1.2.5. A term $M \in \Lambda$ is closed if and only if $\mathbf{FV}(M) = \emptyset$, closed lambda terms are often referred to as combinators. Λ^0 is the set of all closed lambda terms.

Note 2. They are called combinators since having no free variables means that all variables are bound, and thus, a closed term just combines bound variables.

Variable Substitution

We approach the definition for α-equivalence to this end we have to introduce what it means to substitute a variable.

Definition 1.2.6. Capture-Avoiding Substitution Rules Let $M, N \in \Lambda$ and $x, y, z \in \mathcal{V}$. The substitution $M[x := N]$ is defined inductively as:

$$
\begin{aligned}
x[x := N] &= N \\
y[x := N] &= y && y \neq x \\
(M_1\, M_2)[x := N] &= (M_1[x := N])\,(M_2[x := N]) \\
(\lambda x.M)[x := N] &= \lambda x.M \\
(\lambda y.M)[x := N] &= \lambda y.(M[x := N]) && y \neq x, y \notin \mathbf{FV}(N) \\
(\lambda y.M)[x := N] &= \lambda z.(M[y := z][x := N]) && y \neq x, y \in \mathbf{FV}(N)
\end{aligned}
$$

where $z \notin \mathbf{FV}(M) \cup \mathbf{FV}(N)$. Implementing the latter in Haskell requires a bit more code than previous definitions, along with making some practical tradeoffs:

```
capAvoidSubs :: Term -> String -> Term -> Term
capAvoidSubs same@(Var y) x n
  | y == x = n
  | otherwise = same
capAvoidSubs (App t1 t2) s t3 =
  App (capAvoidSubs t1 s t3) (capAvoidSubs t2 s t3)
capAvoidSubs (Abs y m) x n
  | y == x = (Abs y m)
  | otherwise =
      if Set.member y (freeVars n)
      then
        let z = freshVar y (Set.union (freeVars m) (freeVars n))
        in (Abs z (capAvoidSubs (capAvoidSubs m y (Var z)) x n))
      else
        (Abs y (capAvoidSubs m x n))
  where
    freshVar :: String -> Set.Set String -> String
    freshVar s terms =
      if Set.member (s ++ "'") terms
      then
        freshVar (s ++ "'") terms
      else
        s ++ "'"
```

Note 3. `freshVar` generates a new name for binding variables such that $z \notin \mathbf{FV}(M) \cup \mathbf{FV}(N)$. In this instance we have chosen to add single quotes as long as the variable name is contained in $\mathbf{FV}(M) \cup \mathbf{FV}(N)$.

Hopefully the meaning of the first three rules is clear, they ancapsulate renaming in the context of variables and applications. The subsequent rules are about abstractions, rule four states that renaming a binding variable has no effect, renaming only modifies a term whenever the renamed variable is different from the binding variable. Rule five is the general case, that is, when the substitution just moves in together with the abstracted term. Then there is the last case, when the binding variable is a bound occurrence in the term we are substituting, which poses a problem, to see why, let us work through an example:

Example 1.2.3. If we mistakenly substitute x by $(\lambda w.\, w\, y)$ in $(\lambda y.\, y\, x)$ using rule five, we get:

$$(\lambda y.\, y x)[x := (\lambda w.\, w\, y)] = (\lambda y.\, y\, (\lambda w.\, w\, y))$$

This is problematic since the free variable y became bound after the rename. Free and bound variables should maintain their freedom or boundedness after a rename. The correct way to substitute avoiding the name clash:

$$(\lambda y.\, y x)[x := (\lambda w.\, w\, y)] = (\lambda z.\, z\, (\lambda w.\, w\, y))$$

Binding variable serve as placeholders, therefore, renaming should not affect the integrity of an expression.

Note 4. Hence the use of Capture Avoiding alongside Substitution.

One notable result regarding substitution is the substitution lemma:

Lemma 1.2.1. Let M, N, P be lambda terms and x, y be variables. Then:

$$M[x := N][y := P] = \begin{cases} M[x := N[y := P]], & \text{if } x = y, \\ M[y := P][x := N[y := P]], & \text{if } x \neq y \text{ and } y \notin \mathrm{FV}(N). \end{cases}$$

Proof. By structural induction on M, using α-conversion to avoid capture and assuming bound variables x, y, N, P are fresh.

- **Variable case:** Direct from the definition of substitution; check $z = x$, $z = y$, and z distinct from both.
- **Application case:** Substitution distributes:

  $$(M_1 M_2)[x := N] = M_1[x := N]\, M_2[x := N],$$

  so both statements follow immediately from the induction hypothesis on M_1 and M_2.

- **Abstraction case:** With z fresh,

$$(\lambda z.M_0)[x := N] = \lambda z.(M_0[x := N]),$$

  and the induction hypothesis applies to M_0.

The side condition in (2) ensures the $[y := P]$ substitution does not enter N when changing the order. □

In the previous example a bound variable is renamed to avoid a naming collision and avoid binding a previously free variable, α-equivalence is the generalization of this technique, it defines equivalence up to renaming of binding variables.

Notation 2. In many of the upcoming definitions inference rules are used, here is a simple introduction for those unfamiliar with the concept:

$$\frac{P_1 \quad P_2 \quad \cdots \quad P_n}{C}\ \textsc{Name}$$

where P_i are judgments assumed to hold, C is the judgment inferred from the premises and NAME is an optional identification label.

α-equivalence

Definition 1.2.7. Two lambda terms $M, N \in \Lambda$ are α-equivalent, written $M =_\alpha N$, if they are structurally identical except for the names of bound variables. Formally:

$$\frac{}{\lambda x.L =_\alpha \lambda y.L[x := y]}\ \textsc{Alpha} \qquad \frac{L =_\alpha L'}{\lambda x.L =_\alpha \lambda x.L'}\ \textsc{Abs}$$

$$\frac{L_1 =_\alpha L_1'}{L_1\,L_2 =_\alpha L_1'\,L_2}\ \textsc{AppL} \qquad \frac{L_2 =_\alpha L_2'}{L_1\,L_2 =_\alpha L_1\,L_2'}\ \textsc{AppR}$$

α-equivalence is an equivalence relation:

(Reflexivity)	$M =_\alpha M$
(Transitivity)	$M =_\alpha N$ and $N =_\alpha P \Rightarrow M =_\alpha P$
(Symetry)	$M =_\alpha N \Rightarrow N =_\alpha M$

Example 1.2.4. A simple equivalence class and two samples of not α-equivalent pairs.

$$\lambda x.\lambda y.\,y =_\alpha \lambda a.\lambda b.\,b =_\alpha \lambda z.\lambda x.\,x$$

$$\lambda x.\lambda y.\,x \neq_\alpha \lambda x.\lambda y.\,y \quad \lambda x.\,x\,y \neq_\alpha \lambda x.\,x\,z$$

Remember terms are α-equivalent only when bounded names are properly substituted.

Remark 3. α-equivalence comes naturally once substitution is defined, since the names of bound variables are just placeholders that will eventually hold an argument, thus getting replaced.

Note 5. Variable naming is a nightmare. De Bruijn indexing is a way of methodically assigning numbers to variable names, indicating how many binders away a variables binder is. This system eliminates the need for α equivalence to exist.

$$\lambda x.x \Rightarrow \lambda.0 \quad \lambda x.\lambda y.x \Rightarrow \lambda.\lambda.1 \quad \lambda x.\lambda y.y \Rightarrow \lambda.\lambda.0$$

We are better at names than we are at numbers, which is why we will prefer using symbols over De Brujn indexing. Programs like automated theorem provers and compilers do use similar naming strategies to simplify their work.

β-equivalence

Defines the means to perform computation in the λ-calculus, applying substitution as a means of reduction for expressions. This is similar to how one simplifies an arithmetic expression, only it is more powerful, since arithmetic expressions lack expressiveness **??**.

Definition 1.2.8. Single-step β-reduction $\rightsquigarrow_\beta$ is defined using the following inference rules:

$$\frac{}{(\lambda x.M)\,N \rightsquigarrow_\beta M[x := N]}\ \textsc{Beta} \qquad \frac{M \rightsquigarrow_\beta M'}{\lambda x.M \rightsquigarrow_\beta \lambda x.M'}\ \textsc{Abs}$$

$$\frac{M_1 \rightsquigarrow_\beta M_1'}{M_1\,M_2 \rightsquigarrow_\beta M_1'\,M_2}\ \textsc{AppL} \qquad \frac{M_2 \rightsquigarrow_\beta M_2'}{M_1\,M_2 \rightsquigarrow_\beta M_1\,M_2'}\ \textsc{AppR}$$

where $M, N \in \Lambda$. In Haskell:

```
betaRed :: Term -> Term
betaRed same@(Var x) = same
betaRed (Abs x y) = Abs x (betaRed y)
betaRed (App (Abs x y) z) = capAvoidSubs y x z
betaRed (App x y) = App (betaRed x) (betaRed y)
```

Single β-reductions can be applied succesively, which induces the following definition

Definition 1.2.9. Multi-step β-reduction. We write $M \rightsquigarrow_\beta^* N$ if and only if there exists a sequence of one-step β-reductions starting from M and ending at N.[2]

$$M \equiv M_0 \rightsquigarrow_\beta M_1 \rightsquigarrow_\beta M_2 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta M_{n-1} \rightsquigarrow_\beta M_n \equiv N$$

[2]The convention in the literature is to use $\rightarrow_\beta$ for β-reductions $\twoheadrightarrow_\beta$ for multi-step β-reductions. I prefer to use $\rightsquigarrow_\beta$ and $\rightsquigarrow_\beta^n$ since this allows me to use the superscript to note the number of steps in the process β-reduction e.g. $(\lambda f.\lambda x.x)\,M\,N \rightsquigarrow_\beta^2$ N. Instead of $\rightsquigarrow_\beta^n$ I use $\rightsquigarrow_\beta^*$ whenever the number of steps is undefined.

That is, there exists an integer $n \geq 0$ and a sequence of terms $M_0, M_1, \ldots, M_n$ such that: $M_0 \equiv M$ and $M_n \equiv N$ for all i with $0 \leq i < n$, we have $M_i \rightsquigarrow_\beta M_{i+1}$.

The process of succesively β-reducing a term leads raises the question of whether all terms can be reduced indefinitly. We know the answer to this, since for instance, variables in $\mathcal{V}$ cannot be reduced. The subsequent question is then, Do all all terms have some kind of normal form that cannot be reduced further? This is answered in Section 1.5. To this end we introduce a formal definition of normal form that will allow us to gain insight into the normalization properties of the λ-calculus.

Definition 1.2.10. A λ-term M is in normal form if there is no term N such that $M \rightsquigarrow_\beta N$. Equivalently, M contains no subterm of the form $(\lambda x.P)\,Q$ that can be reduced by beta-reduction. In other words, M is irreducible under beta-reduction.

Definition 1.2.11. A reducible expression, or redex for short, is any subterm of the form $(\lambda x.P)\,Q$. Such a term can be reduced by a single beta-reduction step, resulting in $P[x := Q]$.

Definition 1.2.12. β-equivalence relation, equivalence of λ-terms through β-reductions.

$$M =_\beta N \iff M \rightsquigarrow_\beta^* N \vee N \rightsquigarrow_\beta^* M$$

Note 6. $=_\beta$ is the symetric closure of $\rightsquigarrow_\beta^*$.

We now explore a key aspect of the λ-calculus, in the jargon referred to as first-class citizenship, or high-order, a property of terms that restrains them from being exclusively classified as either function or argument. The example below sheds some light on the matter:

Example 1.2.5. High Order by example. Let us β-reduce the λ-term ABC and inspect the consequences:

$$\begin{aligned}
\overbrace{(\lambda f.\lambda y.f\,f\,y)}^{A}\overbrace{(\lambda x.x+1)}^{B}\overbrace{(2)}^{C} &\rightsquigarrow_\beta (\lambda y.(\lambda x.x+1)(\lambda x.x+1)\,y)(2) \\
&\rightsquigarrow_\beta (\lambda x.x+1)(\lambda x.x+1)(2) \\
&\rightsquigarrow_\beta^* (\lambda x.x+1)(3) \\
&\rightsquigarrow_\beta^* (4)
\end{aligned}$$

Taking a closer look, A, applies f twice to x, as a consequence, B is being applied twice to C. And so, very easily, we have implemented the $x+2$ function via a sequential application of $x+1$ to C.

Remark 4. As seen above, we can pass what we understand as conventional functions as arguments since they are treated alike. One can intuitively apreciate the computational expresivenes this brings with it, and how this syntactic-semantic homogeneity sets the λ-calculus apart from the classical set-theoretic approach to functions.

η-equivalence

This last notion of equivalence, refers to a program being equivalent to itself wrapped with an identity function:

Definition 1.2.13. Two lambda terms M and N are said to be η-equivalent, written $M =_\eta N$, if:

$$\frac{x \notin \mathbf{FV}(M)}{(\lambda x.\, M\, x) =_\eta M}\ \textsc{Eta} \qquad \frac{M =_\eta M'}{\lambda x.\, M =_\eta \lambda x.\, M'}\ \textsc{Abs}$$

$$\frac{M_1 =_\eta M_1'}{M_1\, M_2 =_\eta M_1'\, M_2}\ \textsc{AppL} \qquad \frac{M_2 =_\eta M_2'}{M_1\, M_2 =_\eta M_1\, M_2'}\ \textsc{AppR}$$

The abstraction $\lambda x.\,(M\ x)$ performs the same operation as M, so if x is not used freely within M, the abstraction serves no purpose. This is shown in Eta, the rest of the inference rules propagate this equivalence throughout the whole term.

Note 7. Two terms are observationally equivalent if they cannot be distinguished by any context that can be papplied to them. η-equivalence can be understood as a form of weak extensionality, since it does not fully encode observational equivalence.

$\beta\eta$-reduction

To later prove the Church–Rosser property, we introduce a set of rules that captures reduction taking extensionality into account, we combine both β and η-reduction into a single relation.

Definition 1.2.14. We write $M \rightsquigarrow_{\beta\eta} N$ if M reduces to N by one application of either a β or an η-rule, possibly in a subterm. The rules are:

$$\frac{}{(\lambda x.M)\, N \ \rightsquigarrow_{\beta\eta}\ M[x := N]}\ \textsc{Beta} \qquad \frac{x \notin \mathbf{FV}(M)}{\lambda x.(M\, x)\ \rightsquigarrow_{\beta\eta}\ M}\ \textsc{Eta}$$

$$\frac{M_1 \rightsquigarrow_{\beta\eta} M_1'}{M_1\, M_2\ \rightsquigarrow_{\beta\eta}\ M_1'\, M_2}\ \textsc{AppL} \qquad \frac{M_2 \rightsquigarrow_{\beta\eta} M_2'}{M_1\, M_2\ \rightsquigarrow_{\beta\eta}\ M_1\, M_2'}\ \textsc{AppR}$$

$$\frac{M \rightsquigarrow_{\beta\eta} M'}{\lambda x.M\ \rightsquigarrow_{\beta\eta}\ \lambda x.M'}\ \textsc{Abs}$$

Definition 1.2.15. We write $M \rightsquigarrow^*_{\beta\eta} N$ if M reduces to N by zero or more single-step $\beta\eta$-reductions, the same way we did for β-reduction in 1.2.9.

1.3 Normalization & The Church-Rosser Theorem

In this section we attempt to give the reader an overview of the properties behind this calculus. We analyze what makes the untyped λ-calculus a suitable computation model, and provide enough evidence that it is missing some desirable properties that motivate the introduction of typing.

The Church-Rosser in the λ-Calculus

To this end we specify some tools general to relations, specifically, those that help us reason about confluence and termination.

Definition 1.3.1. Let $R \subseteq A \times A$ be a binary relation on a set A. The reflexive-transitive closure of R, denoted by R^*, is the smallest relation on A such that:

$$\begin{array}{ll} \text{(Reflexivity)} & (x,x) \in R^* \\ \text{(Transitivity)} & (x,y) \in R^* \wedge (y,z) \in R^* \Rightarrow (x,z) \in R^* \\ \text{(Inclusion of } R\text{)} & (x,y) \in R \Rightarrow (x,y) \in R^* \end{array}$$

where $x, y, z \in A$.

Remark 5. R^* contains all pairs that relate to each other in such a way that x can be transformed into y by applying R zero or more times. Essentially, reflexive transitive closures are chains of relations that relate terms from the first to the last of the chain if there is one.

Example 1.3.1. $\rightsquigarrow_\beta^*$ defined in 1.2.9 is the reflexive transitive closure of $\rightsquigarrow_\beta$. Equivalently, $\rightsquigarrow_{\beta\eta}^*$ is too the reflexive transitive closure of $\rightsquigarrow_{\beta\eta}$.

Definition 1.3.2. Let $\rightarrow$ be a binary relation on a set A. The symbol $\rightarrow^*$ denotes reflexive-transitive closure of $\rightarrow$. We define the Church–Rosser poperty:

$$x \rightarrow^* y \wedge x \rightarrow^* z \Rightarrow \exists w \in A,\ y \rightarrow^* w \wedge z \rightarrow^* w$$

where $x, y, z, w \in A$. In a diagram:

$$\begin{tikzcd} & x \arrow[dl, "*"'] \arrow[dr, "*"] & \\ y \arrow[dr, dashed, "*"'] & & z \arrow[dl, dashed, "*"] \\ & w & \end{tikzcd}$$

The following section covers the proof of this property for the untyped λ-calculus. We deemed it necessary to include the proof since it provides evidence that the λ-calculusis a viable system to perform computation with, since it ensures that no matter what reduction path we choose first the reduction can always converge into a unique term. See 1.3.2. This notion of confluence is strengthened when we introduce typing 2.3.

Remark 6. If you have no time to waste, Nagele, Oostrom, and Sternagel 2016 is a mechanized proof of the Church-Rosser Theorem using the Z-property written in Nominal Isabelle, which enables you to swiftly glance over the following section.

Proof of the Church-Rosser for $\beta\eta$-Reduction

In this section, we prove the Church–Rosser Theorem for $\beta\eta$-reduction as done in Selinger 2008, even though proof is due to Tait and Martin-Löf. We begin by defining a new relation $M \rightrightarrows M'$ on terms, called parallel one-step reduction. It is the smallest relation satisfying the following rules:

$$\frac{P \rightrightarrows P' \qquad N \rightrightarrows N'}{PN \rightrightarrows P'N'} \ \textsc{ParApp}(1) \qquad\qquad \frac{N \rightrightarrows N'}{\lambda x.N \rightrightarrows \lambda x.N'} \ \textsc{ParAbs}(2)$$

$$\frac{Q \rightrightarrows Q' \qquad N \rightrightarrows N'}{(\lambda x.Q)N \rightrightarrows Q'[N'/x]} \ \textsc{ParBeta}(3) \qquad\qquad \frac{P \rightrightarrows P' \qquad x \notin \mathbf{FV}(P)}{\lambda x.(P\,x) \rightrightarrows P'} \ \textsc{ParEta}(4)$$

The relation $\rightrightarrows$ is also reflexive, $x \rightrightarrows x$ for every x.

Theorem 1.3.1. The Church-Rosser Theorem states that if $M \rightsquigarrow_\beta^* N_1$ and $M \rightsquigarrow_\beta^* N_2$, then there exists P such that

$$N_1 \rightsquigarrow_\beta^* N_3 \quad \text{and} \quad N_2 \rightsquigarrow_\beta^* N_3.$$

Proof. Any β-reduction can be simulated by parallel reduction. Suppose $M \Longrightarrow_\beta N_1$ and $M \Longrightarrow_\beta N_2$. By lemma **??**,

$$N_1 \Longrightarrow_\beta M_\beta^* \quad \text{and} \quad N_2 \Longrightarrow_\beta M_\beta^*.$$

Parallel reduction implies ordinary multi-step β-reduction: $N_1 \rightsquigarrow_\beta^* M_\beta^*$ and $N_2 \rightsquigarrow_\beta^* M_\beta^*$. Hence M_β^* is a common reduct. □

Remark 7. The untyped λ-calculus does not obey the Diamond property. The proof of this fact is given later on when we present fixed points and recursion **??**.

Corollary 1.3.2. If a λ-term M reduces to two normal forms N_1 and N_2, then they are unique up to alpha-equivalece:

$$M \rightsquigarrow_\beta^* N_1 \ \text{ and } \ M \rightsquigarrow_\beta^* N_2 \ \text{ and } N_1, N_2 \text{ are normal } \ \Rightarrow \ N_1 =_\alpha N_2$$

Proof. By the Church-Rosser theorem, there exists a term P such that $N_1 \rightsquigarrow_\beta^* P$ and $N_2 \rightsquigarrow_\beta^* P$. Since N_1 and N_2 are normal forms, they cannot be reduced further. Thus, the only possible P is $N_1 =_\alpha N_2$. □

Looking at the big picture, we find this normalization property to be a non-trivial result that helps us verify that the λ-calculus is a deterministic model of computation. Meaning that if we take a program and run it, it will always give the same output for a given input. Of course, when we say a program, we mean a λ-term, and when we say run, we mean β-reduce. Perhaps the following section enables the reader to better grasp why we informally speak of lambda terms as programs.

Normalization

Normalization properties essentially summarize the termination properties of terms in the context of β-reduction.

Definition 1.3.3. Consider a lambda term M and all possible β-reduction sequences

$$M \equiv M_0 \rightsquigarrow_\beta M_1 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta M_n$$
$$M \equiv N_0 \rightsquigarrow_\beta N_1 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta$$
$$\cdots$$

We classify M as follows:

- M is strongly normalizing if every such reduction sequence terminates in a normal form. Equivalently, there is no infinite reduction sequence.

$$M \equiv M_0 \rightsquigarrow_\beta M_1 \rightsquigarrow_\beta M_2 \rightsquigarrow_\beta \cdots$$

- M is weakly normalizing if there exists at least one finite reduction sequence that terminates in a normal form.

$$M \equiv M_0 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta M_n$$

- M is non-normalizing if no reduction sequence starting from M ever reaches a normal form. In this case, every reduction path is either infinite or loops without producing a normal form.

Note 8. Applying our newly found knowledge on Church–Rosser, we know that out of those different reduction sequences, there always exists some other reduction sequence such that the sequences cross paths.

Proposition 1.3.3. A term may be either, weakly normalizing, strongly normalizing or non-normalizing.

Proof. Examples are:

- $\lambda x.x$ is strongly normalizing since it is already in normal form.
- $\Omega = (\lambda x.xx)(\lambda x.xx)$ reduces to itself forever, hence non-normalizing.

- $(\lambda x.y)\,\Omega \rightsquigarrow_\beta y$, so it has a normal form y, but reducing Ω loops, so it is weakly normalizing.

□

Remark 8. Checking whether a term is terminating is undecidable in general, since it is an instance of the halting problem.

1.4 Relevant Constructions

This section presents a number of conventional constructions that represent key ideas common to computer science and other fields. We are specifically referring to natural numbers, branching statements, booleans and lists among others, as well as the operations over them. These constructions can be used to prove relevant statements concerning the expressiveness of the λ-calculus.

Note 9. We use our Haskell program to bring some computer science constructions to life, which means that some of the details in this section are implementation dependent. In addition, we use the following type aliases to simplify notation:

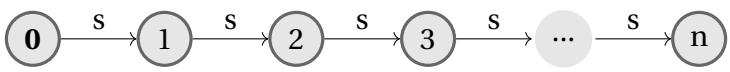
```
v :: String -> Term
v = Var
l :: String -> Term -> Term
l = Abs
a :: Term -> Term -> Term
a = App
```

Natural Numbers

Definition 1.4.1. Peano Axioms, the set of natural numbers $\mathbb{N}$

- **(Zero)** $0 \in \mathbb{N}$
- **(Successor)** $\forall n \in \mathbb{N}, S(n) \in \mathbb{N}$
- (Initial) $\forall n \in \mathbb{N}, S(n) \neq 0$
- (Injectivity) $\forall n, m \in \mathbb{N}, S(n) = S(m) \Rightarrow n = m$
- (Induction) $P(0) \wedge \forall (P(n) \Rightarrow P(S(n))) \Rightarrow \forall n, P(n)$

The first four axioms can be summarized using a graph:



Example 1.4.1. Say for instance we mean to represent 3 using Peano notation, using the graph, we would come up with $s(s(s(0)))$, since it is 3 steps away from 0.

Definition 1.4.2. To add and multiply natural numbers we use following the rules:

$$n+m=\begin{cases} n & \text{if } m=0 \\ s(n+m') & \text{if } m=s(m') \end{cases} \qquad a\cdot b=\begin{cases} 0 & \text{if } b=0 \\ a\cdot b'+a & \text{if } b=s(b') \end{cases}$$

- **a + b**: A recursive rule that performs the addition, and a base case to stop the recursion for n + 0, since there is no such number that has 0 as succesor.
- **a · b**: Using addition we define multiplication in a similar way, adding a as many times as the function s is applied to 0 in b.

Definition 1.4.3. Representation of the set of all naturals in the λ-calculus, usually referred to as Church numerals:

$$\overline{0} \equiv \lambda f.\lambda x.x$$
$$\texttt{SUCC} \equiv \lambda n.\lambda f.\lambda x.f(nfx)$$

Using our Haskell program, $\overline{0}$ is represented like follows:

```
zero  = l "f" (l "x" (v "x"))
```

Example 1.4.2. The church numerals have been defined similarly to the peano axioms, that is, by bringing into existance an initial element and using the succesor function to define the rest of them. Let us apply SUCC a few times to illustrate:

$$\overline{0} \equiv \lambda f.\lambda x.x$$
$$\overline{1} \equiv \lambda f.\lambda x.fx$$
$$\overline{2} \equiv \lambda f.\lambda x.f(fx)$$
$$\overline{3} \equiv \lambda f.\lambda x.f(f(fx))$$
$$\overline{\mathrm{n}} \equiv \lambda f.\lambda x.f^n x$$

where n denotes the number of times we append f.

Notation 3. We use $\overline{\text{number}}$ to denote a number is church encoded.

In our Haskell program we use `nat` to convert from Haskell's prelude type `Natural` to type our custom type `Term`.

```
nat :: Natural :: Term
nat n = normalize (nat_ n)
    where
        nat_ 0 = zero
        nat_ n = a succ_ (nat_ (n - 1))
```

Remark 9. Since λ-terms are first-class citizens, and f and x are abstracted away, the reality is that numbers are a quite general construct that has many distinct applications. Recall Example 1.2.5 where $A \equiv_\alpha \overline{2}$.

Basic Arithmetic

Definition 1.4.4. Now that we have a construction for the naturals, we can define some arithmetic operations:

$$\begin{aligned}
\texttt{ADD} &\equiv \lambda m.\lambda n.\lambda f.\lambda x.mf(nfx) \\
\texttt{MUL} &\equiv \lambda f.\lambda x.\lambda f.\lambda x.m(nf)x \\
\texttt{ISZERO} &\equiv \lambda n.n\,(\lambda x.\texttt{FALSE})\,\texttt{TRUE} \\
\texttt{PRED} &\equiv \lambda n.\lambda f.\lambda x.\big(((n\,(\lambda g.\lambda h.h\,(g\,f)))\,(\lambda u.x))\,(\lambda u.u)\big)
\end{aligned}$$

PRED computes the predecessor of a church encoded natural. $\texttt{PRED}\,\overline{0} = \overline{0}$. We omit further Haskell implementation of constructions from now on since they are just literal translations to our programs representation of λ-terms.

Example 1.4.3. Now we illustrate how addition works by performing $2+3$ with the system we defined.

$$\begin{aligned}
\texttt{ADD}\,2\,3 &\equiv (\lambda m.\lambda n.\lambda f.\lambda x.m\,f\,(n\,f\,x))\,(\lambda f.\lambda x.f\,f\,x)\,(\lambda f.\lambda x.f\,f\,f\,x) \\
&\leadsto_\beta^* \lambda f.\lambda x.\,(\lambda f.\lambda x.f\,f\,x)\,f\,((\lambda f.\lambda x.f\,f\,f\,x)\,f\,x) \\
&\leadsto_\beta^* \lambda f.\lambda x.\,(\lambda x.f\,f\,x)\,((\lambda x.f\,f\,f\,x)\,x) \\
&\leadsto_\beta (\lambda f.\lambda x.\,f\,f\,f\,f\,f\,x) \\
&\equiv 5
\end{aligned}$$

In Haskell:

```
ghci> normalize (a (a mul (nat 2)) (nat 3))
```

This unequivocally yields `(λf.(λx.f(f(f(f(f(fx))))))).`

Remark 10. Multiplication works similarly to addition. As for ISZERO, the intuition behind it is that if the number is zero, then $\lambda x.\texttt{FALSE}$ is dropped, and the result is TRUE; otherwise, it is applied at least once, which immediately yields FALSE.

Remark 11. This proves that the λ-calculus is capable of arithmetic.

Booleans & Branching

The next constructions in line are the truth values of boolean logic, "*true*" and "*false*" for short.

Definition 1.4.5. Boolean truth values:

$$\begin{aligned}
\texttt{TRUE} &\equiv \lambda t.\lambda f.t \\
\texttt{FALSE} &\equiv \lambda t.\lambda f.f
\end{aligned}$$

Remark 12. Note how $\texttt{TRUE}\ M\ N \leadsto_\beta M$ and $\texttt{FALSE}\ M\ N \leadsto_\beta N$

Definition 1.4.6. Conditional, in other words *"if statement"*.

$$\texttt{IF} \equiv \lambda b.\lambda t.\lambda e.b\,t\,e$$

The IF statement is just a combinator that happens to behave similarly to a branching statement whenever we pass a church encoded boolean as a first argument.

Example 1.4.4. On the correct behabiour of IF i.e. check that it implements branching. $M, N \in \Lambda$ and IF, TRUE, FALSE are as usual:

$$\begin{aligned} &\texttt{IF}\,\texttt{TRUE}\,M\,N \\ \equiv\ &(\lambda b.\lambda t.\lambda e.\,b\,t\,e)\,\texttt{TRUE}\,M\,N \\ \rightsquigarrow_\beta\ &(\lambda t.\lambda e.\texttt{TRUE}\,t\,e)\,M\,N \\ \rightsquigarrow_\beta\ &(\lambda e.\texttt{TRUE}\,M\,e)\,N \\ \rightsquigarrow_\beta\ &\texttt{TRUE}\,M\,N \rightsquigarrow_\beta^* M \end{aligned}$$

$$\begin{aligned} &\texttt{IF}\,\texttt{FALSE}\,M\,N \\ \rightsquigarrow_\beta^*\ &\texttt{FALSE}\,M\,N \rightsquigarrow_\beta^* N \end{aligned}$$

So the IF just combines b, t, e in such a way that whenever b is a church encoded boolean either t or e is dropped depending on the truth value of b. See Remark 12. From this example we extract that $\lambda x.x$ is equivalent to IF it terms of behaviour, and thus, we could use $\texttt{IF} \equiv \lambda x.x$ interchangeably.

Remark 13. Using the IF we can implement an universal set of logic gates:

$$\begin{aligned} \texttt{NOT} &\equiv \texttt{IF}\,b\,\texttt{FALSE}\,\texttt{TRUE} \\ \texttt{AND} &\equiv \texttt{IF}\,a\,(\texttt{IF}\,b\,\texttt{TRUE}\,\texttt{FALSE})\,\texttt{FALSE} \end{aligned}$$

This allows us to simulate a nand gate, known to be universal. Thus, the lambda calculus is at least equivalent to propositional logic.

Pairs

A fundamental construct for building data structures is the notion of pairs. They allow us to bind two values together and query them independently.

Definition 1.4.7. $\texttt{PAIR} \equiv \lambda x.\lambda y.\lambda f.f\,x\,y$

Remark 14. This definition is convenient since

$$\texttt{PAIR}\,M\,N \rightsquigarrow_\beta^* \lambda f.f\,M\,N$$

which means we can retrieve the first component via $(\lambda f.f\,M\,N)\,\texttt{TRUE}$ and the second via $(\lambda f.f\,M\,N)\,\texttt{FALSE}$. This motivates the following projection operators:

Definition 1.4.8. A PAIR is nothing but an ordered relation $\langle A, B\rangle \in A \times B$. The projections π_1, π_2 are defined as:

$$\begin{aligned} \texttt{FST} &\equiv \lambda p.p\,(\lambda x.\lambda y.x) \\ \texttt{SND} &\equiv \lambda p.p\,(\lambda x.\lambda y.y) \end{aligned}$$

Proposition 1.4.1. A key property of PAIR is:

$$\texttt{PAIR}\,A\,B \equiv_\beta \texttt{PAIR}\,(\texttt{FST}\,(\texttt{PAIR}\,A\,B))\,(\texttt{SND}\,(\texttt{PAIR}\,A\,B))$$

Lists

We move on to how we can now define linear lists.

Definition 1.4.9. We define the empty list in the same spirit as 0 and FALSE:

$$\texttt{NIL} \equiv \lambda f.\lambda z.z$$

It represents the terminating element of the inductive definition of lists. To build non-empty lists we use the constructor:

$$\texttt{CONS} \equiv \lambda h.\lambda t.\lambda f.\lambda z.f\,h\,(t\,f\,z)$$

which binds a new element h to a pre-existing list t or to NIL.

Definition 1.4.10. Some other relevant list operators are:

$$\begin{aligned}
\texttt{ISNIL} &\equiv \lambda l.l\,(\lambda h.\lambda t.\texttt{FALSE})\,\texttt{TRUE}\\
\texttt{HEAD} &\equiv \lambda l.((l\,(\lambda h.\lambda t.h))\,\texttt{NIL})\\
\texttt{TAIL} &\equiv \lambda l.\texttt{FST}(l\,(\lambda h.\lambda t.\texttt{PAIR}\,t\,t)\,(\texttt{PAIR}\,\texttt{NIL}\,\texttt{NIL}))
\end{aligned}$$

1.5 Fixed Point Combinators and Recursion

The Church Rosser Property labels the untyped λ-calculus as a confluent system, therefore we know that whenever a term has a normal form it must be unique. We still have not given an answer to whether all λ-terms have a normal form.

The most straight forward way to test this, is to check whether there exists some term that reduces to itself, effectively creating a loop in the β-reduction chain so to speak. In more formal terms this means there exists some M such that $M \rightsquigarrow_\beta^n M$ with $n > 0$. It turns out $\Omega \equiv (\lambda x.\,x\,x)(\lambda x.\,x\,x)$ fullfills this requirement. Let us inspect what goes on while β-reducing this term:

$$\Omega \equiv (\lambda x.\,x\,x)(\lambda x.\,x\,x) \rightsquigarrow_\beta (\lambda x.\,x\,x)(\lambda x.\,x\,x) \rightsquigarrow_\beta^* (\lambda x.\,x\,x)(\lambda x.\,x\,x) \equiv \Omega$$

This so called Ω-combinator, proves that not all terms have a normal form, and shines a beam of light onto how recursion can be achieved in the λ-calculus.

There is one caveat to this, that classically, recursion is achieved through naming, take for instance this definition for the gdc:

$$\underline{\text{gcd}}(a,b) = \begin{cases} a & b = 0 \\ \underline{\text{gcd}}(b,\, a \bmod b) & b \neq 0 \end{cases}$$

Here, when $b \neq 0$ the name gcd is used to refer to the same definition again. In the λ-calculus such self-referencing definitions are not permited since the language does not allow names.

Fixed Points

To bypass this setback the notion of fixed point is of particular interest.

Definition 1.5.1. Let $f : X \to X$ be a function. An element $x \in X$ is called a fixed point of f if $f(x) = x$.

Example 1.5.1. Some simple functions that illustrate what fixed points are:

Function	*Fixed Point(s)*
$f(x) = c$	$x = c$
$g(x) = x$	$\{x \mid x \in \mathbb{R}\}$
$h(x) = x^2$	$x = 0,\ x = 1$

This harmless idea satisfies an interesting property, namely that $x = f(x) = f(f(x)) = f^n(x)$ effectively creating a chain where every element $f^{n+1}(x) = f(f^n(x))$, attaining self-reference without naming and showcasing the ability of fixed points to induce recursion. Our quest now, is to find a term that simulates this behavior. To do this, let us translate the definition for fixed point into the world of λ-calculus.

Definition 1.5.2. Given a lambda term $L \in \Lambda$, a term $M \in \Lambda$ is called a fixed point of L if $M =_\beta LM$.

No surprises here, the definition is close to a one-to-one mapping of its functional counterpart seen previously. Interestingly enough, it turns out that every lambda term has a fixed point:

Theorem 1.5.1. For every $L \in \Lambda$, there exists $M \in \Lambda$ such that M is a fixed point for M i.e. $M =_\beta LM$. Referred to as the Fixed Point Theorem in the literature.

Proof. Given $L \in \Lambda$, define M as below:

$$M := (\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x))$$

M is a reducible expression:

$$\begin{aligned} M &\equiv (\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x)) \\ &\rightsquigarrow_\beta L((\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x))) \\ &\equiv L\,M. \end{aligned}$$

Hence, $M =_\beta LM$, as required. □

Remark 15. See how LM is the application of M to L and that using APPR we can keep β-reducing M the same way we did for $x = f(x) = f(f(x))\ldots$ only this time using lambda terms:

$$M \rightsquigarrow_\beta L\,M \rightsquigarrow_\beta L\,L\,M \rightsquigarrow_\beta^* L^n M.$$

Recursive Combinators

Now that nameless self-reference has been proven to exist within the the λ-calculus, we take a step forward and introduce a few of the most important fixed point combinators and explore their different natures.

Definition 1.5.3. Y Combinator. This is the term that given some other term returns its fixed point.

$$Y \triangleq \lambda f.\,(\lambda x.\, f\,(x\,x))\,(\lambda x.\, f\,(x\,x))$$

Remark 16. Even though the Y combinator is of theoretical interest since it allows us to find recursion within the λ-calculus, it is of little practical under strict evaluation since it always runs into non-termination e.g.

$$\begin{aligned}
Y\,F &\equiv (\lambda f.\,(\lambda x.\, f\,(x\,x))(\lambda x.\, f\,(x\,x)))F \\
&\rightsquigarrow_\beta (\lambda x.\, F\,(x\,x))(\lambda x.\, F\,(x\,x)) \\
&\rightsquigarrow_\beta F((\lambda x.\, F\,(x\,x))(\lambda x.\, F\,(x\,x))) \\
&\rightsquigarrow_\beta F(F((\lambda x.\, F\,(x\,x))(\lambda x.\, F\,(x\,x)))) \\
&\rightsquigarrow_\beta \cdots
\end{aligned}$$

Under strict evaluation, the argument F is not evaluated until Y is done unraveling i.e. never.

To work around this issue, several other combinators exist, the Z and the Turing combinators are two of the better known ones:

Definition 1.5.4. $Z \triangleq \lambda f.\,(\lambda x.\, f\,(\lambda v.\, x\,x\,v))\,(\lambda x.\, f\,(\lambda v.\, x\,x\,v))$

Example 1.5.2. Unfolding the Z Combinator

$$\begin{aligned}
Z\,F &\equiv (\lambda f.\,(\lambda x.\, f\,(\lambda v.\, x\,x\,v))\,(\lambda x.\, f\,(\lambda v.\, x\,x\,v)))\,F \\
&\rightsquigarrow_\beta (\lambda x.\, F\,(\lambda v.\, x\,x\,v))\,(\lambda x.\, F\,(\lambda v.\, x\,x\,v)) \\
&\rightsquigarrow_\beta F\,(\lambda v.\,(\lambda x.\, F\,(\lambda v.\, x\,x\,v))\,(\lambda x.\, F\,(\lambda v.\, x\,x\,v))\,v) \\
&\equiv F\,(\lambda v.\, Z\,F\,v)
\end{aligned}$$

Remark 17. The extra $\lambda v.\cdot$ in the final form of $Z\,F$ is essential to delay evaluation in a strict environment. Without it, the recursive call would be applied immediately, and cause non-termination in a call-by-value setting.

Definition 1.5.5. Turing combinator $\Theta \triangleq (\lambda x.\,\lambda y.\, y\,(x\,x\,y))\,(\lambda x.\,\lambda y.\, y\,(x\,x\,y))$

We now use the Turing combinator to aid us implement an untyped version of the factorial function.

λ-Calculus Implementation of Factorial

Classically, factorial is defined as $n! = \prod_{k=1}^{n} k$, that has the following recursive interpretation $n! = n \cdot (n-1)!$ with $0! = 1$. This section develops a worked out example of how to implement this in the λ-calculus.

Example 1.5.3. Implementing factorial using the acquired toolkit:

$$\underline{\texttt{fact}} = \lambda n.\,\texttt{if then else}\,(\texttt{iszero}\,n)\,(\overline{1})\,(\texttt{mult}\,n\,(\underline{\texttt{fact}}\,(\texttt{pred}\,n))),$$

As already seen, the problem with a recursive definition such as this one is that the name `fact`, appears both on the left and the right-hand side. This means finding `fact` requires solving an equation, which we will do thanks to fixed points. We start by abstracting away `fact` from the term:

$$\texttt{fact} = \overbrace{\left(\lambda f.\lambda n.\,\texttt{if then else}\,(\texttt{iszero}\,n)\,(1)\,(\texttt{mult}\,n\,(f\,(\texttt{pred}\,n)))\right)}^{F}\,\texttt{fact}.$$

Then the last equation becomes $\texttt{fact} = F\,\texttt{fact}$, which is a fixed point equation. We can solve it up to β-equivalence by letting $\texttt{fact} = \Theta F$. Using our Haskell program we define `fact`:

```
fact = a turing
    (l "f" (l "n"
        (a (a (a if_ (a iszero (v "n"))) (nat 1))
        (a (a mul (v "n"))
            (a (v "f")
            (a pred_ (v "n")))))
    ))
```

We run the following command to compute $\texttt{fact}\,\overline{3}$:

```
ghci> normalize (a fact (nat 3))
```

That of course yields $\lambda f.\lambda x.\,f(f(f(f(f(f\,x))))) \equiv \overline{6}$ as result.

Note 10. The latter implementation exhibits a striking syntactic resemblance to that of the **LISP** family of languages.

Conclusions

We have covered some of the most important notions that surround the untyped λ-calculus. It has been shown to satisfy the Church-Rosser property of confluence, it has also been shown that this property is not the strongest of its kind. We have also seen that it is not strongly normalizing. For this reasons, it is in our best interest to add some spice to the λ-calculus by means typing. This is to try and make it comply with some stronger requirements that will make it more practical to perform computation. We covered untyped λ-calculus the next chapters focuses on minimal typing variations.

- **Untyped**: Where the type of expressions is never specified. As we have seen, this gives us maximal expressiveness at the cost of being unsafe. Informally, this means we are at a risk of composing unrelated terms that produce miscalculations. For instance xM.
- **Minimal Typing**: Every expression is annotated with a type. This would be equivalent to the usual set theoretic approach to computation where functions have well defined domain and codomain.
- **Extended Typing**: Similar to minimal typing but extended with constructions like products and sums.

This alternatives have dramatically different properties from one another. We will cover them in some detail throughout the text.

References and Resources. This chapter is influenced by many of the classic resources on the subject, primarily Nederpelt and Geuvers 2014 for the basic aspects of the λ-calculus. The proof to the Church-Rosser Theorem and the factorial example are fine-tuned versions of what can be found in Selinger 2008, though, for what it's worth, the factorial implementation is original. Similar proofs are found in O. L. Project 2025 and Nagele, Oostrom, and Sternagel 2016. Other text used as general resources are Barendregt and Barendsen 2000 and Girard 1989. We intentionally skipped some of the topics in the literature like combinatory algebras that are not that relevant to our focus of study.

Chapter 2

Typing

We covered the untyped λ-calculus, highlighting its strengths and limitations. This section introduces the notion of typing, which restricts its expressiveness while improving the tractability of studying its properties. The idea of typing arises from the usual definition of a function, understood as a mapping between elements of sets. For example, consider a function $f : A \to B$ defined by $a \mapsto f(a)$, where we recognize A as the domain and B as the codomain. Notice how the mapping is meaningful only within this context–otherwise f would lack sense. Hence, the important of restricting what a function can take as an input as well as what it may yield as an output.

In the context of the λ-calculus, types serve as a means of restriction, similar ho how we use logical predicates to define sets. Just as $f : ? \to ?$ is refined to $f : A \to B$ by specifying requirements on the arguments and ensuring properties of the outputs, we use types to impose requirements on the arguments of an abstraction and to ensure properties of the reduction process of that term.

Since the λ-calculus treats all terms as first-class citizens, attempting to impose restrictions directly on arguments is not effective. Instead, we define restrictions on all terms by pairing each term with additional information called a *type*. This tells us about the nature of a given term, which may be of two kinds: a simple type or arrow type $\to$. Introducing a typing system into the λ-calculus splits Λ into two categories: Typable and Non-typable terms. These sets are disjoint. This simple type system classifies expressions such as $x(\lambda y.y)$ or Y as untypable, while $\lambda x.x$ has type $\alpha \to \alpha$.

In the following we present minimal and extended simple types as well as the different i.e. Church and Curry style.

2.1 Minimal Typing

The untyped version of the λ-calculus relies on abstraction and application. Minimal typing captures this simply by using base types and arrow types.

Definition 2.1.1. The set of minimal simple types $\mathcal{T}$ is defined inductively as follows:

- (Type variable) If $\alpha \in \mathscr{B}$, then $\alpha \in \mathscr{T}$.
- (Arrow type) If $\sigma, \tau \in \mathscr{T}$, then $(\sigma \to \tau) \in \mathscr{T}$.

where $\mathscr{B}$ is a countably infinite set of base types. In abstract syntax:

$$\mathscr{T} ::= \mathscr{B} \mid (\mathscr{T} \to \mathscr{T})$$

Note 11. These minimal types are the most basic types one can apply to the λ-calculus, this is with respect to extended types, that use products and sums to add expressiveness.

Notation 4. In general, greek letters $\alpha, \beta, \gamma, \ldots$ are standard to denote type names.

Notation 5. The convention is that $\to$ is right associative, that is: $\alpha \to \gamma \to \alpha$ is the same as $(\alpha \to (\gamma \to \alpha))$.

Example 2.1.1. Examples of simple types include:

$$\gamma \quad (\beta \to \gamma) \quad ((\gamma \to \alpha) \to (\alpha \to (\beta \to \gamma)))$$

Definition 2.1.2. A context Γ is a finite mapping from variables to simple types, defined inductively by:

$$\Gamma ::= \cdot \mid \Gamma, x : \sigma$$

where

- $\cdot$ is the empty context
- x is a variable
- $\sigma \in \mathscr{T}$ is a simple type
- $\Gamma, x : \sigma$ extends Γ by mapping x to σ
- $\mathrm{dom}(\Gamma) = \{x \mid x : \sigma \in \Gamma\}$
- $\mathrm{rng}(\Gamma) = \{\sigma \mid x : \sigma \in \Gamma\}$

A context is often represented as a finite sequence: $\Gamma = x_1 : \sigma_1, x_2 : \sigma_2, \ldots, x_n : \sigma_n$ with all x_i distinct.

With this in mind we distinguish two alternative methods for typing. Typing à la Church, where types are annotated alongside λ-terms. And Typing a la Curry, where typing judgements are provided externally through some inference algorithm. Church's style of typing adds syntax on top of the already defined λ-calculus. It can be said that while Church's typing introduces rules to generate well-typed terms, Curry's approach infers types from untyped expressions.

- Curry-style:

$$\lambda x.x \quad \text{with} \quad \vdash \lambda x.x : \sigma \to \sigma$$

- Church-style:

$$\lambda x:\sigma.x \quad \text{with} \quad \vdash \lambda x:\sigma.x:\sigma\to\sigma$$

Note how Curry's style allows us to assign different valid types to terms. For example, $(\alpha\to\beta)\to(\alpha\to\beta)$ could be a valid type assignment for $\lambda x.x$, do not confuse this polymorphism.

Typing à la Curry

Definition 2.1.3. We define the following typing relation:

$$\frac{x\notin \mathrm{dom}(\Gamma)}{\Gamma, x:\tau\vdash x:\tau}\,\textsc{Var} \qquad \frac{x\notin \mathrm{dom}(\Gamma)\quad \Gamma, x:\sigma\vdash M:\tau}{\Gamma\vdash \lambda x.M:\sigma\to\tau}\,\textsc{Abs}$$

$$\frac{\Gamma\vdash M:\sigma\to\tau \qquad \Gamma\vdash N:\sigma}{\Gamma\vdash MN:\tau}\,\textsc{App}$$

Example 2.1.2. In this example we verify that is a valid type for using Curry style of typing.

$$\frac{\dfrac{\Gamma, x:\sigma\vdash \lambda x.\,f\,x:\tau}{\Gamma\vdash \lambda f.\lambda x.\,f\,x:(\alpha\to\alpha)\to(\alpha\to\alpha)}\,\textsc{Abs} \qquad \dfrac{\dfrac{}{\Gamma, y:\alpha\vdash y:\alpha}\,\textsc{Var}}{\Gamma\vdash \lambda y.\,y:\alpha\to\alpha}\,\textsc{Abs}}{\vdash (\lambda f.\lambda x.\,f\,x)(\lambda y.\,y):\alpha\to\alpha}\,\textsc{App}$$

Note 12. Notice that we can replace α by any other basic type and the derivation tree would still hold. Moreover, we could replace y by a different term with a different type and the structure of the tree would hold as long as we also replace α with the corresponging type.

Example 2.1.3. Let σ,τ,ρ be arbitrary types. Then:

- $\vdash \lambda x.x:\sigma\to\sigma$
- $\vdash \lambda x.\lambda y.x:\sigma\to\tau\to\sigma$
- $\vdash \lambda x.\lambda y.\lambda z.x\,z\,(y\,z):(\sigma\to\tau\to\rho)\to(\sigma\to\tau)\to\sigma\to\rho$

Definition 2.1.4. If $\Gamma\vdash M:\sigma$, then we say that M has type σ in Γ. A term M is typable if there exist Γ and σ such that $\Gamma\vdash M:\sigma$.

Typing à la Church

...

For the rest of the text we will consider typing a la church.

2.2 Extended Typing

As discussed, minimal typing is the simplest form of typing. It follows from this statement that it can be extended. Here we introduce products that can be thought of as records or structs:

Definition 2.2.1. Extended types $\mathscr{T}^\times$ is defined inductively as:

$$\mathscr{T}^\times ::= \mathscr{B} \mid \mathscr{T}^\times \to \mathscr{T}^\times \mid \mathscr{T}^\times \times \mathscr{T}^\times \mid 1$$

where $\mathscr{B}$ is the set of base types.

Definition 2.2.2. The set of raw terms $\Lambda^\times$ is defined by:

$$\Lambda^\times ::= x \mid \lambda x : \mathscr{T}.\ \Lambda^\times \mid \Lambda^\times\ \Lambda^\times \mid \langle \Lambda^\times, \Lambda^\times \rangle \mid \pi_1 \Lambda^\times \mid \pi_2 \Lambda^\times \mid *$$

Here:

- $x \in \mathscr{V}$, a countable set of variables
- $\lambda x : \mathscr{T}.\ \Lambda^\times$ is function abstraction with type annotation
- $\langle \Lambda^\times, \Lambda^\times \rangle$ is the term constructor for product values
- π_1 and π_2 are projections on product terms
- $*$ is the unique term of type 1

Definition 2.2.3. Given $t \in \Lambda^\times$, typing judgments are of the form $\Gamma \vdash t : \mathscr{T}$, where Γ is a context $x_1 : T_1, \ldots, x_n : T_n$. The typing rules for $\Lambda^\times$ are:

$$\frac{}{\Gamma \vdash * : 1}\ \textsc{Unit} \qquad \frac{}{\Gamma, x : A \vdash x : A}\ \textsc{Var}$$

$$\frac{\Gamma \vdash m : a \to b \qquad \Gamma \vdash n : a}{\Gamma \vdash m\,n : b}\ \textsc{app} \qquad \frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x : A.\ M : A \to B}\ \textsc{Abs}$$

$$\frac{\Gamma \vdash M : A \qquad \Gamma \vdash N : B}{\Gamma \vdash \langle M, N \rangle : A \times B}\ \textsc{Pair} \qquad \frac{\Gamma \vdash M : A \times B}{\Gamma \vdash \pi_1 M : A}\ \textsc{Proj}_1 \qquad \frac{\Gamma \vdash M : A \times B}{\Gamma \vdash \pi_2 M : B}\ \textsc{Proj}_2$$

Reduction. The operational semantics extends β-reduction:

$$(\lambda x : A.\ t)\ u \to t[x := u], \quad \pi_1(t, u) \to t, \quad \pi_2(t, u) \to u.$$

Kinds of problem in Type Theory

These are the usual problems type theorists face. A programmer should be at least familiar with typability, and even more so with Type Checking, the one responsible for the daily headaches of bug finding in typed languages:

- **Typability**: Determine whether a term is legal. Formally, given a term, find a context and type such that $? \vdash \texttt{term} : ?$ holds. If the term is not well-typed, identify where the derivation fails. Type Assignment is a variant where the context is given, so the task is only to find the type: $\Gamma \vdash \texttt{term} : ?$
- **Type Checking**: Given a context, term, and type, check whether $\Gamma \vdash \texttt{term} : \texttt{type}$ holds. The task is purely to verify that the term has the specified type in the given context.
- **Inhabitation**: Given a context and a type, determine whether there exists a term with that type: $\Gamma \vdash ? : \texttt{type}$

Remark 18. In programming languages and theorem provers, these type problems have concrete meanings. Well-typedness asks whether a term or program is valid under the typing rules, ensuring correctness of proofs or programs. Type Checking verifies that a given term has a specified type, allowing compilers or proof assistants to confirm correctness. Term Finding constructs a term of a given type, corresponding to automatic proof generation in theorem provers or program synthesis in programming languages.

2.3 Normalization in Minimal and Extended Typing

Chapter 3

Intuitionistic Logic & Curry–Howard

Unlike classical logic, which is often extended through modal operators or second-order quantifiers, intuitionistic logic takes a different path, not because it adds power, but because it restricts classical reasoning. Classical logic freely allows non-constructive arguments, proofs that assert the existence of an object without providing a way to explicitly construct it. Intuitionistic logic, in contrast, is designed to formalize constructive reasoning, where existence is meaningful only if a specific example can be provided.

3.1 Intuitionistic Logic

To motivate the introduction of Intuitionistic Logic, we present a theorem and two different styles of proof T. O. L. Project 2025:

Theorem 3.1.1. There are irrational numbers a and b such that a^b is rational.

Proof. Let $a, b = \sqrt{2}$

- If $\sqrt{2}^{\sqrt{2}}$ is rational, we are done.
- Otherwise, $\sqrt{2}^{\sqrt{2}}$ is irrational and we can consider $a = \sqrt{2}^{\sqrt{2}}$ and $b = \sqrt{2}$. In which case:
$$a^b = \left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} = \sqrt{2}^{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2}^2 = 2,$$
which is rational so a and b must exist.

□

This is a perfectly valid proof. However, it is somewhat unsatisfying, since it proves existence without exhibiting a specific pair. A more constructive proof is as follows

Proof. Take $a = \sqrt{3}$ and $b = \log_3 4$. Then

$$a^b = \left(\sqrt{3}\right)^{\log_3 4} = 3^{\frac{1}{2}\log_3 4} = \left(3^{\log_3 4}\right)^{1/2} = 4^{1/2} = 2,$$

which is rational. □

Syntax

The syntax for intuitionistic logic is the same as that of propositional logic. Where $\vee, \wedge, \rightarrow$ are used as binary connectives to compose logical statements and $\neg$ is used to negate such statements.

Definition 3.1.1. The set of formulas of propositional intuitionistic logic is defined as follows:

- $\perp$ is an atomic formula.
- Every propositional variable p_i is a formula.
- If φ and ψ are formulas, then $(\varphi \wedge \psi)$ is a formula.
- If φ and ψ are formulas, then $(\varphi \vee \psi)$ is a formula.
- If φ and ψ are formulas, then $(\varphi \rightarrow \psi)$ is a formula.

In addition to the connectives introduced above, the following symbols define the following symbols to simplify notation:

- $\neg\varphi$ abbreviates $(\varphi \rightarrow \perp)$.
- $\varphi \leftrightarrow \psi$ abbreviates $(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$.

The Brouwer–Heyting–Kolmogorov Interpretation

The Brouwer–Heyting–Kolmogorov Interpretation interpretation provides an informal, constructive understanding of intuitionistic connectives. It is based on the notion of a construction, which can be regarded as a constructive proof. The BHK interpretation assigns meanings to the intuitionistic connectives as follows:

- We assume that we know what constitutes a construction for an atomic proposition.
- A construction of $\varphi_1 \wedge \varphi_2$ is a pair $\langle M_1, M_2 \rangle$, where M_1 is a construction of φ_1 and M_2 is a construction of φ_2.
- A construction of $\varphi_1 \vee \varphi_2$ is a pair $\langle s, M \rangle$, where $s = 1$ and M is a construction of φ_1, or $s = 2$ and M is a construction of φ_2.

- A construction of $\varphi \to \psi$ is a function that converts any construction of φ into a construction of ψ.
- There is no construction for $\bot$; it represents absurdity.
- Defined as $\varphi \to \bot$. Thus, a construction of $\neg\varphi$ is a function that converts a construction of φ into a construction of $\bot$.

Remark 19. Generally speaking, $\neg\varphi$ means a construction of φ is impossible.

Natural Deduction

It is a deduction system within intuitionistic logic that uses constructive reasoning to derive conclusions. This structured proofs emphasize the existence of explicit constructions rather than abstract truth values.

The rules of natural deduction for intuitionistic propositional logic (without $\bot C$) can be presented as follows:

Introduction / Elimination Rules

$$\frac{\phi \qquad \psi}{\phi \wedge \psi}\ \textsc{IntroConj} \qquad \frac{\phi \wedge \psi}{\phi}\ \textsc{ElimConj} \qquad \frac{\phi \wedge \psi}{\psi}\ \textsc{ElimConj}$$

$$\frac{\phi}{\phi \vee \psi}\ \textsc{IntroDisj} \qquad \frac{\psi}{\phi \vee \psi}\ \textsc{IntroDisj} \qquad \frac{\phi \vee \psi \qquad [\phi]^n \cdots \chi \qquad [\psi]^n \cdots \chi}{\chi}\ \textsc{ElimDisj}$$

$$\frac{[\phi] \cdots \psi}{\phi \to \psi}\ \textsc{IntroImpl} \qquad \frac{\phi \qquad \phi \to \psi}{\psi}\ \textsc{ElimImpl}$$

$$\frac{[\phi]^n \cdots \bot}{\neg\phi}\ \textsc{IntroNeg} \qquad \frac{\phi \qquad \neg\phi}{\bot}\ \textsc{ElimNeg}$$

$$\frac{\bot}{\phi}\ \textsc{ElimBot}$$

Example 3.1.1. some deduction example

3.2 The Curry-Howard Isomorphism

The λ-calculus, as introduced in the first section, provides a minimal but expressive framework for modeling computation. Independently, intuitionistic logic offers a formal system for constructive reasoning that requires the existence of

witnesses for propositions. The Curry-Howard correspondence reveals a structural equivalence between them, providing a unifying perspective that connects proofs with programs. The Curry–Howard correspondence establishes the following associations:

- **Propositions ↔ Types**: Proposition in intuitionistic logic can be interpreted as a type in a typed λ-calculus.
- **Proofs ↔ Programs**: Constructive proofs of a proposition correspond to a λ-term inhabiting the associated type.
- **Proof normalization ↔ Program evaluation**: The process of simplifying proofs corresponds to the process of evaluating a term through β-reduction.

Given a proposition P in intuitionistic logic, constructing a proof of P is equivalent to constructing a term t such that $t : T$, where T is the type corresponding to P. This yields the principle often summarized as *"proofs-as-programs"* or *"propositions-as-types"*. The specifics of this relation are summarized in table 3.1.

Table 3.1: The Curry–Howard Correspondence

Intuitionistic Logic	**Typed λ-Calculus**
Proposition A	Type A
Proof of A	Term of type A
Implication $A \to B$	Function type $A \to B$
Conjunction $A \wedge B$	Product type $A \times B$
Disjunction $A \vee B$	Sum type $A + B$
$\top$	1
$\bot$	0
Proof normalization	β-reduction

Example 3.2.1. Consider the implication $A \to B$ in intuitionistic logic. A proof of $A \to B$ is a construction that, given a proof of A, produces a proof of B. Under the Curry–Howard correspondence, this directly maps to a function type $A \to B$ in λ-calculus, where a λ-term $f : A \to B$ takes an argument of type A and returns a result of type B. Similarly, conjunction $A \wedge B$ corresponds to a product type $A \times B$, and disjunction $A \vee B$ corresponds to a sum type $A + B$. In each case, the rules for constructing and deconstructing proofs mirror the operations on the corresponding λ-terms.

Chapter 4

Introduction to Category Theory

Category Theory abstracts the notion of internal structure and studies mathematical objects indirectly through their connections to other objects. In practical terms, instead of studying objects in isolation, category theory shifts the focus onto morphisms among objects, and instead of studying mathematical entities from a definitional stand point, it emphasizes the importance of how constructions relate to one another, keeping us away from manipulating objects directly, instead it uses universality to pinpoint specific properties by uniqueness of morphisms.

Category Theory offers extreme generality and has great power to unify seemingly unrelated fields. It can serve to create blueprints that describe mathematical environments and their properties, and to give structure to patterns that repeat within different fields of mathematics. In some instances, category theory serves as a medium for transporting mathematical statements –such as theorems or lemmas–between domains. A categorical formulation of such results can often be translated to other contexts via functors or equivalences of categories.

One of the fields that benefits from the practical applications of categorical reasoning is computer science, deeply related to category theory through type theory, intuitionistic logic and natural deduction.

4.1 Fundamentals of Category Theory

Let us begin our dive into categories by defining what they are:

Definition 4.1.1. A category $\mathscr{C}$ consists of:

- A collection of objects $\mathrm{Ob}(\mathscr{C})$
- A collection of arrows or morphisims where each arrow f has a domain object $\mathrm{dom}(f)$ and a codomain object $\mathrm{cod}(f)$. These can be defined using $f : A \to B$ to signify $\mathrm{dom}(f) = A$ and $\mathrm{cod}(f) = B$
- A composition operator assigning to each pair of arrows f and g, with $cod(f) = dom(g)$, a composite arrow $g \circ f : \mathrm{dom}(f) \to \mathrm{cod}(g)$ satisfying

the following associative law:

$$h \circ (g \circ f) = (h \circ g) \circ f$$

where $f : A \to B, g : B \to C, h : C \to D$ are arrows of a category

- For each object A, an identity arrow $id_A : A \to A$ satisfying the following identity law:

$$\mathrm{id_B} \circ f = f \quad f \circ \mathrm{id_A} = f$$

for any $f : A \to B$

Remark 20. Literature on the topic usually differentiates between small and large categories. Small categories are those whose collection of objects and whose collection of arrows are both sets. Large categories allow for collections of objects or arrows that are proper classes rather than sets. We will only be concerned with small categories.

Various mathematical constructs fit the description of a category. For example, the category **Set** consists of sets as objects and functions as morphisms, where composition is ordinary function composition and identities are identity functions. Another example is the category **Grp**, whose objects are groups and whose morphisms are group homomorphisms. Other examples are:

Example 4.1.1. Let $\mathscr{C}$ denote a category, consisting of a set of objects, $\mathbf{Ob}(\mathscr{C})$, and a set of arrows, $\mathbf{Hom}(\mathscr{C})$, satisfying the category axioms.

- **empty category**, denoted by $\mathbf{0}$, is the category with no objects and no morphisms.

$$\mathrm{Ob}(\mathbf{0}) = \emptyset, \quad \mathrm{Hom}(\mathbf{0}) = \emptyset$$

- **terminal category** (or **point category**), denoted by $\mathbf{1}$, is the category with a single object and a single morphism, which must be the identity morphism on that object. Formally,

$$\mathrm{Ob}(\mathbf{1}) = \{*\}, \quad \mathrm{Hom}(\mathbf{1}) = \{\mathrm{id}_*\}$$

where $\mathrm{id}_* : * \to *$ is the identity morphism on $*$.

- **category 2**, often representing the poset $A \leq B$, is a category with two objects and three morphisms. Formally,

$$\mathrm{Ob}(\mathbf{2}) = \{A, B\}, \quad \mathrm{Hom}(\mathbf{2}) = \{\mathrm{id}_A, \mathrm{id}_B, f\}$$

where $\mathrm{id}_A : A \to A$ and $\mathrm{id}_B : B \to B$ are identity morphisms, and $f : A \to B$ is a unique non-identity morphism. The compositions are defined as $f \circ \mathrm{id}_A = f$ and $\mathrm{id}_B \circ f = f$.

The most elemental category is the category that has no elements and no arrows, it is commonly represented with a 0. Category 1 contains one object and one identity arrow, as required by the definition. Category 2 contains three arrows, the identities and one arrow from 1 to 2. Category 3 builds up on this as is observed in the following diagram:

$$\begin{tikzcd} 1 \arrow[loop right, "id_1"] & & \\ id_1 \arrow[loop left] \; 1 \arrow[r] & 2 \arrow[loop right, "id_2"] & \end{tikzcd} \qquad \begin{tikzcd} & 2 \arrow[loop right, "id_2"] \arrow[d] & \\ id_1 \arrow[loop left] \; 1 \arrow[ur] \arrow[r] & 3 \arrow[loop right, "id_3"] & \end{tikzcd}$$

Example 4.1.2. In this example we show that **Set** is a category where objects correspond to sets $A, B, C, \ldots$ and arrows to functions. For $f : A \to B$ and $g : B \to C$, composition of functions is $(g \circ f)(a) = g(f(a))$, which is a function $g \circ f : A \to C$. Composition is associative since for $h : C \to D$ we have that: $h \circ (g \circ f)(a) = h(g(f(a))) = (h \circ g)(f(a)) = (h \circ g) \circ f(a)$. Lastly, each set A, the identity morphism is $\mathrm{id}_A(a) = a$, satisfying $f \circ \mathrm{id}_A = f = \mathrm{id}_B \circ f$. Therefore, **Set** is a category. □

Example 4.1.3. In this example we show that any partially ordered set $(P, \leq)$ can be viewed as a category **Pos** whose objects are elements of P and there is a unique arrow $p \to q$ if and only if $p \leq q$. Composition is defined by transitivity: if $p \leq q$ and $q \leq r$, then $p \leq r$, so the composite $p \to r$ exists and is unique. Associativity holds automatically since there is at most one arrow between any two objects. Lastly, each $p \in P$ has an identity arrow $p \to p$ that corresponds to the reflexivity property of partial orders. Therefore, every poset defines a category. □

Example 4.1.4. Let us consider a simple functional programming language with the following types:

$$T ::= \mathrm{Nat} \mid \mathrm{Bool} \mid T \times T \mid T \to T$$

The type system is closed under product and function type constructors. A valid compound type is $(\mathrm{Nat} \times \mathrm{Bool}) \to \mathrm{Nat}$. A categorical interpretation of this scenario reveals types are objects and functions morphisms.

Identity $\mathrm{id}_A : A \to A$ can be defined as $\mathrm{id}_A = \lambda x{:}A.\, x$, which is well-typed. Identity laws are verified trivially: $\mathrm{id}_B \circ f = f = f \circ \mathrm{id}_A$.

Compositionality is defined as sequential application of functions that share domain and codomain through $f : A \to B$ and $g : B \to C$ like $g \circ f : A \to C$ by $g \circ f = \lambda x{:}A.\, g(f(x))$, which is well-typed by function application.

Composition is associative due to β-equivalence in lambda calculus:

$$h \circ (g \circ f) = \lambda x{:}A.h(g(f(x))) = (h \circ g) \circ f$$

Therefore, the language satisfies all axioms of a category.

Example 4.1.5. **Hask** is a category where the objects are Haskell types such as `Int`, `Bool`, `String`. The morphisms are total Haskell functions between those types such as `f::Int->Bool`. For every type `a`, there is an identity morphism `id::a->a` that maps each value to itself. Morphisms can be composed using `(.)::(b->c)->(a->b)->a->c`,

meaning the output of one function becomes the input of another. This composition is associative, and the identity morphisms act as neutral elements for composition. These properties types as objects, functions as morphisms, `id` as identity, and `(.)` as composition make Hask a category.

Category	Description
Set	Sets as objects, functions as morphisms.
Grp	Groups and group homomorphisms.
Ring	Rings and ring homomorphisms.
$\mathbf{Mod}_R$	Modules over a ring R and R-linear maps.
$\mathbf{Vect}_k$	Vector spaces over field k and linear maps.
Top	Topological spaces and continuous functions.
Man	Smooth manifolds with smooth maps.
Hilb	Hilbert spaces and bounded linear maps.

Table 4.1: Key mathematical categories.

Category	Description
Mon	Monoids and monoid homomorphisms.
CPO	Complete partial orders and continuous maps.
Dom	Domains as posets with Scott-continuous maps.
Rel	Sets and binary relations as morphisms.
Pfn	Sets and partial functions as morphisms.
Par	Sets and partial maps (generalized morphisms).
$\mathbf{Kl}(T)$	Kleisli category of a monad T on a base category.
$\mathbf{Endo}(\mathbf{Set})$	Endofunctors on the category of sets.

Table 4.2: Key categories in computer science.

Aside from defining categories from mathematical objects, categores can be built from other categories. One way to do this is via opposite categories.

Definition 4.1.2. For a category $\mathscr{C}$, the opposite or dual category $\mathscr{C}^{op}$ is defined by:

- $\mathrm{Ob}(\mathscr{C}^{op}) = \mathrm{Ob}(\mathscr{C})$,
- For $A, B \in \mathrm{Ob}(\mathscr{C})$, $\mathscr{C}^{op}(A, B) = \mathscr{C}(B, A)$,
- Composition is reversed: if $f : A \to B$ and $g : B \to C$ in $\mathscr{C}$, then

$$f^{op} \circ g^{op} = (g \circ f)^{op}.$$

Hom-set

Definition 4.1.3. In a category $\mathscr{C}$, for any two objects $A, B \in \mathbf{Ob}(\mathscr{C})$, the hom-set from A to B is the set of all morphisms from A to B. This set is denoted by $\mathbf{Hom}(A, B)$.

$$\mathbf{Hom}(A, B) = \{f \mid f\colon A \to B \text{ is a morphism in } \mathscr{C}\}$$

Monos & Epis

Definition 4.1.4. Let $g_1, g_2 : X \to A$ (resp. $h_1, h_2 : B \to Y$). An arrow $f : A \to B$ in $\mathscr{C}$ is a monomorphism (resp. epimorphism) if:

$$f \circ g_1 = f \circ g_2 \implies g_1 = g_2 \quad (\text{resp. } h_1 \circ f = h_2 \circ f \implies h_1 = h_2)$$

Note 13. Monomorphisims generalize the notion of injectivity, while epimorphisms are the categorical generalization of surjectivity.

Functors

Categories provide a way of representing mathematical structures using only objects and arrows. Category theory takes itself very seriously, and since its focus is not on the objects themselves but on the relations between them, it is natural to consider functors, which are themselves morphisms in the category of categories. In this way, category theory can study itself.

With this perspective, the definition of a functor arises naturally as a pair of functions that map both objects and arrows from one category to another while preserving identities and composition. Formally:

Definition 4.1.5. Let $\mathscr{C}$ and $\mathscr{D}$ be categories. A functor $F : \mathscr{C} \to \mathscr{D}$ consists of:

- A function that maps objects from $\mathscr{C}$ to $\mathscr{D}$:

$$\begin{aligned} F : \mathrm{Ob}(\mathscr{C}) &\longrightarrow \mathrm{Ob}(\mathscr{D}) \\ X &\longmapsto F(X) \end{aligned}$$

- A function that maps arroes in $\mathscr{C}$ to $\mathscr{D}$:

$$\begin{aligned} F : \mathrm{Hom}_{\mathscr{C}}(X, Y) &\longrightarrow \mathrm{Hom}_{\mathscr{D}}(F(X), F(Y)) \\ f &\longmapsto F(f) \end{aligned}$$

such that the properties of a category are kept across the transformation:

- For every object $X \in \mathscr{C}$, $F(\mathrm{id}_X) = \mathrm{id}_{F(X)}$.
- For all composable morphisms $f : X \to Y$ and $g : Y \to Z$ in $\mathscr{C}$,

$$F(g \circ f) = F(g) \circ F(f).$$

The following diagram elegantly summarizes the latter definition:

$$\begin{tikzcd} X \arrow[r, "f"] \arrow[d, mapsto, "F"'] & Y \arrow[d, mapsto, "F"] \\ F(X) \arrow[r, "F(f)"'] & F(Y) \end{tikzcd}$$

Example 4.1.6. Haskell captures the notion of functor using the following type class:

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
    (<$) :: a -> f b -> f a
    (<$) = fmap . const
```

There exist several instances for this class, among which we find:

```
instance Functor [] where
    fmap = map
```

That allows us to perform mapping operations on lists like:

```
ghci> fmap (*2) [1,2,3]
[2,4,6]
```

Example 4.1.7. Consider the categories of groups **Grp** and sets **Set**. We define the functor $U : \mathbf{Grp} \longrightarrow \mathbf{Set}$:

- **On objects**: for each group $G \in \mathbf{Grp}$, set $U(G)$ to be the set of elements of G.
- **On morphisms**: for each group homomorphism $f : G \to H$, we define $U(f) = f$, now considered as a function $U(G) \to U(H)$ between sets.

This mapping preserves identities and composition. Diagrammatically, we can represent the action of U on a morphism $f : G \to H$ as:

$$\begin{tikzcd} G \arrow[r, "f"] \arrow[d, mapsto, "U"'] & H \arrow[d, mapsto, "U"] \\ U(G) \arrow[r, "U(f)"'] & U(H) \end{tikzcd}$$

Since this functor "forgets" the structure its domain category we refer to it as forgetful functor.

Example 4.1.8. A reformulation from example 2.1.2 in Pierce 1991 that defines a functor called List from the category **Set** → **Set** to illustrate functors from a programming perspective.

- For every set S, we define a new set:

$$\text{List}(S) := \{L \mid L \text{ is a finite list of belements of } S\}$$

  Where the order of elements matters, and duplicates are allowed.

- For every function $f : S \to S'$, we define a new function

$$\text{List}(f) : \text{List}(S) \to \text{List}(S')$$
$$\text{List}(f)([s_1, s_2, \ldots, s_n]) := [f(s_1), f(s_2), \ldots, f(s_n)].$$

  So, say for instance we have $S = \{1,2\}$ and $f(s) = 2 \cdot s$. Then, $S' = \{2,4\}$, $\text{List}(\{1,2\}) = \{[], [1], [2], [1,1], [1,2], [2,1], [2,2], [1,1,1], \ldots\}$, and $\text{List}(f)(\{2,4\}) = \{[], [2], [4], [2,2], [2,4], [4,2], [4,4], [2,2,2], \ldots\}$.

To confirm that List is a functor, it must satisfy the following two laws:

- **Identity Law:** For every set S and every list $L \in \text{List}(S)$: $\text{List}(\text{id}_S)(L) = L$.
- **Composition Law:** For any two functions $f : S \to S'$ and $g : S' \to S''$, and any list $L \in \text{List}(S)$: $\text{List}(g \circ f)(L) = (\text{List}(g) \circ \text{List}(f))(L)$.

Note 14. $(\text{List}(S), ++, [])$ is a monoid, where $++$ is associative list concatenation and $[]$ is the identity, satisfying $[] + +L = L = L + +[]$.

Natural Transformations

To define natural transformations we will apply the same reasoning scheme as we did with functors. We had categories and we defined functors as morphisms between categories, we do the same with natural transformations, that are nothing but functor morphisms. Suppose we have two functors $F, G : \mathscr{C} \to \mathscr{D}$. These represent two distinct ways of translating $\mathscr{C}$ into $\mathscr{D}$. A natural transformation between F and G then provides a systematic way to relate their outputs. Formally:

Definition 4.1.6. Let $\mathscr{C}$ and $\mathscr{D}$ be categories, and let $F, G : \mathscr{C} \to \mathscr{D}$ be functors. A natural transformation $\alpha : F \Rightarrow G$ between them, denoted:

$$\begin{tikzcd} \mathscr{C} \arrow[r, bend left=60, "F"{name=U}] \arrow[r, bend right=60, "G"'{name=D}] & \mathscr{D} \arrow[Rightarrow, from=U, to=D, "\alpha"] \end{tikzcd}$$

is an assignment to every object $X \in \text{Ob}(\mathscr{C})$ of a morphism $\alpha_X : F(X) \to G(X)$ in $\mathscr{D}$ called the component of α at X, such that for every morphism $f : X \to Y$ in $\mathscr{C}$, the following diagram commutes:

$$\begin{tikzcd} F(X) \arrow[r, "\alpha_X"] \arrow[d, "F(f)"'] & G(X) \arrow[d, "G(f)"] \\ F(Y) \arrow[r, "\alpha_Y"] & G(Y) \end{tikzcd}$$

A Haskell example that makes the diagram commute is:

```
ghci> (reverse . (fmap toUpper)) "abc"
"CBA"
ghci> ((fmap toUpper) . reverse) "abc"
"CBA"
```

This example is one of the simplest of its kind since we are dealing with an endofunctor in **Hask**.

Definition 4.1.7. An endofunctor on a category $\mathscr{C}$ is a functor $F : \mathscr{C} \to \mathscr{C}$ that maps objects and morphisms of $\mathscr{C}$ to objects and morphisms of the same category, preserving identities and composition.

Example 4.1.9. This example is in principle the same as the latter, but may help to grasp the idea of functor in Haskell:

```
data BinTree a
    = Empty
    | Node a (BinTree a) (BinTree a)
    deriving (Show, Eq)

instance Functor BinTree where
    fmap _ Empty = Empty
    fmap f (Node x l r) = Node (f x) (fmap f l) (fmap f r)

reversePairs :: BinTree a -> BinTree a
reversePairs Empty = Empty
reversePairs (Node x l r) = Node x (reversePairs r) (reversePairs l)

testNaturality :: (Eq b) => (a -> b) -> BinTree a -> Bool
testNaturality f t =
    (reversePairs . fmap f) t == (fmap f . reversePairs) t
```

We now can call the `testNaturality` function to assert whether the `f` we provide makes the diagram commute.

```
ghci> f n = 2 * n
ghci> testNaturality f t
True
```

In this case, any function that just operates on the keys will be a valid operation since it does not alter structure.

Note 15. From this example we extract a design pattern. For a given data structure whether a graph, a tree or otherwise, an operation that modifies structure can be considered a natural transform whenever we only deal with functions that don't modify structure.

Note 16. In standard Haskell, we can only create endofunctors since the syntax only allows us to work with types.

Natural transformations, as well as functors can be composed as well, we study now the two ways there are of composing natural transformations:

Remark 21. Given functors $F, G, H : \mathscr{C} \to \mathscr{D}$, natural transformations $\alpha : F \Rightarrow G$ and $\beta : G \Rightarrow H$ can be composed:

$$\beta \circ \alpha : F \Rightarrow H$$

$$\begin{tikzcd}
\mathscr{C} \arrow[r, bend left=60, "F"{name=U}] \arrow[r, bend right=60, "H"'{name=D}] \arrow[r, "G"{name=M}] & \mathscr{D} \arrow[from=U, to=M, Rightarrow, "\alpha"] \arrow[from=M, to=D, Rightarrow, "\beta"]
\end{tikzcd}$$

with components $(\beta \circ \alpha)_X = \beta_X \circ \alpha_X$ for each object $X \in \mathrm{Ob}(\mathscr{C})$.

Remark 22. Given functors $F, G, H : \mathscr{C} \to \mathscr{D}$, natural transformations $\alpha : F \Rightarrow G$ and $\beta : G \Rightarrow H$ can be composed *vertically*:

$$\beta \circ \alpha : F \Rightarrow H$$

$$\begin{tikzcd}
\mathscr{C} \arrow[r, bend left=60, "F"{name=U}] \arrow[r, bend right=60, "H"'{name=D}] \arrow[r, "G"{name=M}] & \mathscr{D} \arrow[from=U, to=M, Rightarrow, "\alpha"] \arrow[from=M, to=D, Rightarrow, "\beta"]
\end{tikzcd}$$

with components

$$(\beta \circ \alpha)_X = \beta_X \circ \alpha_X$$

for each object $X \in \mathrm{Ob}(\mathscr{C})$.

Given categories $\mathscr{C}, \mathscr{D}, \mathscr{E}$, functors $F, G : \mathscr{C} \to \mathscr{D}$ and $J, K : \mathscr{D} \to \mathscr{E}$, and natural transformations $\alpha : F \Rightarrow G$ and $\gamma : J \Rightarrow K$, we can define the *horizontal composition*:

$$\gamma * \alpha : J \circ F \Rightarrow K \circ G$$

with components

$$(\gamma * \alpha)_X = \gamma_{G(X)} \circ J(\alpha_X) = K(\alpha_X) \circ \gamma_{F(X)}$$

for each $X \in \mathrm{Ob}(\mathscr{C})$.

Remark 23. The collection of all functors $F : \mathscr{C} \to \mathscr{D}$ together with natural transformations between them forms the functor category $\mathbf{Fun}(\mathscr{C}, \mathscr{D}) \in \mathrm{Ob}(\mathbf{Cat})$.

Universal Constructions

Definition 4.1.8. Let $\mathscr{C}$ be a category and let $A, B \in \mathrm{Ob}(\mathscr{C})$. A product of A and B is an object $A \times B$ together with morphisms $\pi_1 : A \times B \to A, \pi_2 : A \times B \to B$ such that for every object X with morphisms $f : X \to A$ and $g : X \to B$, there exists a unique morphism $\langle f, g \rangle : X \to A \times B$ making the diagram commute:

$$\begin{tikzcd}
 & X \arrow[dl, "f"'] \arrow[d, dashed, "\langle f,g \rangle"] \arrow[dr, "g"] & \\
A & A \times B \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & B
\end{tikzcd}$$

$$\pi_1 \circ \langle f, g \rangle = f \quad \pi_2 \circ \langle f, g \rangle = g$$

Note 17. In these diagrams, the dashed arrow indicates a unique morphism guaranteed by the universal property of the product or coproduct. That is, for any mediating maps as in the diagram, there exists exactly one morphism making the diagram commute.

Definition 4.1.9. Let $\mathscr{C}$ be a category and let $A, B \in \mathrm{Ob}(\mathscr{C})$. A coproduct of A and B is an object $A + B$ together with morphisms $\iota_1 : A \to A + B, \iota_2 : B \to A + B$ such that for every object X with morphisms $f : A \to X$ and $g : B \to X$, there exists a unique morphism $[f, g] : A + B \to X$ making the diagram commute:

$$\begin{tikzcd}
A \arrow[r, "\iota_1"] \arrow[dr, "f"'] & A + B \arrow[d, dashed, "{[f,g]}"] & B \arrow[l, "\iota_2"'] \arrow[dl, "g"] \\
 & X &
\end{tikzcd}$$

$$[f, g] \circ \iota_1 = f \quad [f, g] \circ \iota_2 = g$$

Chapter 5

Curry–Howard–Lambek & Categorical Semantics

Bibliography

Nadal, Pedro Bonilla (Sept. 2021). "Dependent Typing through Closed Cartesian Categories". Trabajo de Fin de Máster. Madrid: Universidad Autónoma de Madrid.

Barendregt, Henk P. and Erik Barendsen (2000). *Introduction to Lambda Calculus*. Available as PDF. Revised edition (Dec. 1998; Mar. 2000). Lecture notes. URL: https://www.cse.chalmers.se/research/group/logic/TypesSS05/Extra/geuvers.pdf.

Girard, Jean-Yves (1989). *Proofs and Types*. Trans. by Paul Taylor and Yves Lafont. Cambridge Tracts in Theoretical Computer Science 7. Translated and with appendices by Paul Taylor; typeset/appendix by Yves Lafont. Cambridge, UK: Cambridge University Press. ISBN: 9780521371810.

Selinger, Peter (2008). *Lecture Notes on the Lambda Calculus*. Lecture notes / arXiv preprint (revised versions available). Developed from courses taught 2001–2013. URL: https://www.mathstat.dal.ca/~selinger/papers/lambdanotes.pdf.

Sørensen, Morten Heine and Pawe Urzyczyn (2006). *Lectures on the Curry–Howard Isomorphism*. Vol. 149. Studies in Logic and the Foundations of Mathematics. Amsterdam & Boston: Elsevier / North-Holland. ISBN: 9780444520777.

Nederpelt, Rob and Herman Geuvers (2014). *Type Theory and Formal Proof: An Introduction*. Cambridge, UK: Cambridge University Press. ISBN: 9781107036505.

Project, Open Logic (2025). *Parallel beta reduction and the Church Rosser Property*. Revision 91df0ce. URL: https://builds.openlogicproject.org/content/lambda-calculus/church-rosser/parallel-beta-reduction.pdf.

Nagele, Julian, Vincent van Oostrom, and Christian Sternagel (2016). "A Short Mechanized Proof of the Church–Rosser Theorem by the Z-property for the $lambda-beta$-calculus in Nominal Isabelle". In: *arXiv preprint arXiv:1609.03139*. arXiv: 1609.03139 [cs.LO]. URL: https://arxiv.org/abs/1609.03139.

Project, The Open Logic (July 2025). *Intuitionistic Logic*. Revision 91df0ce (2025-07-18). URL: https://builds.openlogicproject.org/content/intuitionistic-logic/intuitionistic-logic.pdf.

Lane, Saunders Mac (1971). *Categories for the Working Mathematician*. 2nd ed. Vol. 5. Graduate Texts in Mathematics. Second edition, expanded, 1998. Springer Science+Business Media.

Pierce, Benjamin C. (1991). *Basic Category Theory for Computer Scientists*. The MIT Press. ISBN: 9780262660716.

Barr, Michael and Charles Wells (1998). *Category Theory for Computing Science*. Revised and available electronically at `http://www.tac.mta.ca/tac/reprints/articles/22/tr22.pdf`. Les Publications CRM, Montreal.